QUICK AND HEAP SORTING : LECTURE

DSAA 211: WEEK 10



QUICK SORTING

Is a highly efficient sorting algorithm and is based on partitioning array of data into smaller arrays and then sort the two resulting subarrays in the same manner.

This algorithm is quite efficient for large-sized data sets as its average and worst-case complexity are O(n2)



HOW QUICK SORT WORKS?



1. Choose a pivot
2. Partition the array around pivot. After partition, it is ensured that all elements are smaller than all right and we get index of the end point of smaller elements. The left and right may not be sorted individually.
3. Recursively call for the two partitioned left and right subarrays.
4. We stop recursion when there is only one element is left.

WHAT IS PIVOT?

- THE PIVOT IS THE “DIVIDER” OR “MIDDLE ELEMENT” THAT QUICK SORT USES TO ORGANIZE THE ARRAY.

The pivot in Quick Sort can be chosen in different ways and yes, it *can* be random, but not always. It depends on how the algorithm is written.

```
//29, 10, 14, 37, 13
```

EXAMPLE:

Step	Current Array / Subarray	Chosen Pivot	Left (Smaller)	Right (Greater)	Result (After Partition)
1	[29, 10, 14, 37, 13]	29	[10, 14, 13]	[37]	[10, 14, 13, 29, 37]
2	[10, 14, 13]	10	[]	[14, 13]	[10, 14, 13]
3	[14, 13]	14	[13]	[]	[13, 14]
4	Combine all	—	—	—	[10, 13, 14, 29, 37]

TIME COMPLEXITY

Feature	Description
Algorithm Type	Divide and Conquer
In-Place	Yes
Stable	No
Best for	Large datasets, when implemented well
Efficiency	Faster than most sorts for big data sets

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

GENERAL ALGORITHM

1. Choose a Pivot
   - Pick one element from the array (can be first, last, middle, or random).
2. Partition the Array
   - Rearrange the array so that:
     - All elements smaller than the pivot go to the left.
     - All elements greater than the pivot go to the right.
3. Place the Pivot
   - After partitioning, the pivot is now in its correct sorted position.
4. Recursively Apply Quick Sort
   - Repeat steps 1–3 on the left and right subarrays.
5. Stop when Subarray Has One Element
   - When there's only one element left, it's already sorted.

PRACTICE

Unsorted List:

`[25, 12, 22, 64, 11]`

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[25, 12, 22, 64, 11]	11	[]	[25, 12, 22, 64]	[**11**, 25, 12, 22, 64]
2	[25, 12, 22, 64]	64	[25, 12, 22]	[]	[11, 25, 12, 22, **64**]
3	[25, 12, 22]	22	[12]	[25]	[11, 12, **22**, 25, 64]

PRACTICE

Unsorted List: [`45, 23, 11, 89, 77, 98, 4, 28, 65]`

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1		65			
2		28			
3		4			
4		11			
5		98			
6		77			

PRACTICE

Unsorted List: [`45, 23, 11, 89, 77, 98, 4, 28, 65]`

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[45, 23, 11, 89, 77, 98, 4, 28, 65]	65	[45, 23, 11, 4, 28]	[89, 77, 98]	[45, 23, 11, 4, 28, **65**, 89, 77, 98]
2	[45, 23, 11, 4, 28]	28	[23, 11, 4]	[45]	[23, 11, 4, **28**, 45, 65, 89, 77, 98]
3	[23, 11, 4]	4	[]	[23, 11]	[**4**, 23, 11, 28, 45, 65, 89, 77, 98]
4	[23, 11]	11	[]	[23]	[4, **11**, 23, 28, 45, 65, 89, 77, 98]
5	[89, 77, 98]	98	[89, 77]	[]	[4, 11, 23, 28, 45, 65, 89, 77, **98**]
6	[89, 77]	77	[]	[89]	[4, 11, 23, 28, 45, 65, **77**, 89, 98]

GAME 1

Unsorted List:

```
[33, 15, 10, 68, 19, 42, 44]
```

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[33, 15, 10, 68, 19, 42, 44]	44			
2		42			
3		19			
4		10			

GAME 1

Unsorted List:

`[33, 15, 10, 68, 19, 42, 44]`

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[33, 15, 10, 68, 19, 42, 44]	44	[33, 15, 10, 19, 42]	[68]	[33, 15, 10, 19, 42, **44**, 68]
2	[33, 15, 10, 19, 42]	42	[33, 15, 10, 19]	[]	[33, 15, 10, 19, **42**, 44, 68]
3	[33, 15, 10, 19]	19	[15, 10]	[33]	[15, 10, **19**, 33, 42, 44, 68]
4	[15, 10]	10	[]	[15]	[**10**, 15, 19, 33, 42, 44, 68]

GAME 2

Unsorted List:

[50, 20, 60, 10, 30, 70, 40]

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[50, 20, 60, 10, 30, 70, 40]	40			
2		30			
3		10			

GAME 2

Unsorted List:

[50, 20, 60, 10, 30, 70, 40]

Step	Current Array / Subarray	Pivot	Left (Smaller than Pivot)	Right (Greater than Pivot)	Result After Partition
1	[50, 20, 60, 10, 30, 70, 40]	40	[20, 10, 30]	[50, 60, 70]	[20, 10, 30, **40**, 50, 60, 70]
2	[20, 10, 30]	30	[20, 10]	[]	[20, 10, **30**, 40, 50, 60, 70]
3	[20, 10]	10	[]	[20]	[**10**, 20, 30, 40, 50, 60, 70]

HEAP SORTING

- HEAP SORT IS A COMPARISON-BASED SORTING ALGORITHM THAT USES A SPECIAL BINARY TREE STRUCTURE CALLED A HEAP TO ARRANGE DATA.

WHAT IS A HEAP?

- A **HEAP** IS A COMPLETE BINARY TREE THAT FOLLOWS A SPECIAL PROPERTY:

Type	Property	Example (Array)
Max Heap	**Parent is greater than or equal to its children**	`[90, 70, 60, 50, 40, 20]`
Min Heap	**Parent is less than or equal to its children**	`[10, 20, 15, 30, 40]`

Heap Sort usually uses a **Max Heap** for **ascending order** sorting.

HOW HEAP SORT WORKS?

- Create a max heap from the given array.
- Swap the root node (the largest element) with the last element in the array.
- Heapify the root node of the current tree to maintain the max heap property.
- Repeat the previous step until the max heap becomes empty, placing elements in their correct positions.
- Continue this process until every element of the array is correctly sorted.

Heapify is the process of arranging elements in an array to maintain the heap property.

EXAMPLE:

6 5 3 1 8 7 2 4

Will Rosenbaum

TIME COMPLEXITY

Feature	Heap Sort
Type	Comparison-based
Technique	Uses binary heap
Best/Worst/Average Time	O(n log n)
Space	O(1)
Stability	Not stable

PRACTICE

```
//29, 10, 14, 37, 13
```

Step	Action	Array
1		
2		
3		
4		
5		
6		
7		
8		

PRACTICE

```
//29, 10, 14, 37, 13
```

Step	Action	Array
1	Build Max Heap	[37, 29, 14, 10, 13]
2	37 ↔ 13	[13, 29, 14, 10, 37]
3	Heapify	[29, 13, 14, 10, 37]
4	29 ↔ 10	[10, 13, 14, 29, 37]
5	Heapify	[14, 13, 10, 29, 37]
6	14 ↔ 10	[10, 13, 14, 29, 37]
7	Heapify	[13, 10, 14, 29, 37]
8	13 ↔ 10	[10, 13, 14, 29, 37]

GAME 1

Unsorted List:

`[45, 12, 88, 32, 5, 60, 19]`

Step	Action	Array State
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

GAME 1

Unsorted List:

[50, 20, 60, 10, 30, 70, 40]

Step	Action	Array State
1	Build Max Heap	[88, 32, 60, 12, 5, 45, 19]
2	Swap 88 ↔ 19	[19, 32, 60, 12, 5, 45, 88]
3	Heapify	[60, 32, 45, 12, 5, 19, 88]
4	Swap 60 ↔ 19	[19, 32, 45, 12, 5, 60, 88]
5	Heapify	[45, 32, 19, 12, 5, 60, 88]
6	Swap 45 ↔ 5	[5, 32, 19, 12, 45, 60, 88]
7	Heapify	[32, 12, 19, 5, 45, 60, 88]
8	Swap 32 ↔ 5	[5, 12, 19, 32, 45, 60, 88]
9	Heapify	[19, 12, 5, 32, 45, 60, 88]
10	Swap 19 ↔ 5	[5, 12, 19, 32, 45, 60, 88]

GAME 2

Unsorted List:

`[42, 17, 8, 23, 56, 34]`

Step	Action	Array State
1		
2		
3		
4		
5		
6		
7		
8		
9		

GAME 2

Unsorted List:

[42, 17, 8, 23, 56, 34]

Step	Action	Array State
1	Build Max Heap	[56, 42, 34, 23, 17, 8]
2	Swap 56 ↔ 8	[8, 42, 34, 23, 17, 56]
3	Heapify	[42, 23, 34, 8, 17, 56]
4	Swap 42 ↔ 17	[17, 23, 34, 8, 42, 56]
5	Heapify	[34, 23, 17, 8, 42, 56]
6	Swap 34 ↔ 8	[8, 23, 17, 34, 42, 56]
7	Heapify	[23, 8, 17, 34, 42, 56]
8	Swap 23 ↔ 17	[17, 8, 23, 34, 42, 56]
9	Heapify	[8, 17, 23, 34, 42, 56]

QUICK AND HEAP SORTING : LABORATORY

DSAA 211: WEEK 9



QUICK SORTING : LABORATORY



DSAA 211: WEEK 10

PRACTICE : QUICK SORT

```
int arr[] = {29, 10, 14, 37, 13};
int n = 5;

printf("Original Array: ");
for (int i = 0; i < n; i++)
printf("%d ", arr[i]);
printf("\n\n");
```

SWAP FUNCTION

```
void swap(int arr[], int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

PARTITION FUNCTION

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choose the last element as pivot
    int i = low - 1;   // Index of smaller element

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr, i, j);
        }
    }
    swap(arr, i + 1, high); // Place pivot in the correct position
    return i + 1; // Return pivot index
}
```

QUICK SORT FUNCTION

```
void quickSort(int arr[], int low, int high, int n) {
    if (low < high) {
        int pi = partition(arr, low, high);

        printf("After partition (Pivot = %d): ", arr[pi]);
        for (int k = 0; k < n; k++)
            printf("%d ", arr[k]);
        printf("\n");

        quickSort(arr, low, pi - 1, n);
        quickSort(arr, pi + 1, high, n);
    }
}
```

CALLS THE SORTING FUNCTION TO SORT THE ENTIRE ARRAY.

```
quickSort(arr, 0, n - 1, n);

    printf("\nSorted Array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
```

```
Original Array: 29 10 14 37 13

After partition (Pivot = 13): 10 13 14 37 29
After partition (Pivot = 29): 10 13 14 29 37

Sorted Array: 10 13 14 29 37
```

HEAP SORTING : LABORATORY

DSAA 211: WEEK 10



PRACTICE : HEAP SORT

```
int arr[] = {29, 10, 14, 37, 13};
    int n = 5;



    printf("Original Array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n\n");
```

HEAPIFY FUNCTION

```
void heapify(int arr[], int n, int i) {
    int largest = i;       //initialize largest as root
    int left = 2 * i + 1;  //left child index
    int right = 2 * i + 2; //right child index
    //if left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;
    //if right child is larger than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    //if largest is not root
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        //recursively heapify the affected subtree
        heapify(arr, n, largest);
    }
}
```

HEAPSORT FUNCTION

```
void heapSort(int arr[], int n) {
    // Step 1: Build a max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    printf("After building Max Heap: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    // Step 2: Extract elements from heap one by one
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        // call max heapify on the reduced heap
        heapify(arr, i, 0);
        printf("After moving max to end: ");
        for (int k = 0; k < n; k++)
            printf("%d ", arr[k]);
        printf("\n");
    }
}
```

CALLS THE SORTING FUNCTION TO SORT THE ENTIRE ARRAY.

```
    heapSort(arr, n);

    printf("\nSorted Array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
```

```
Original Array: 29 10 14 37 13

After building Max Heap: 37 29 14 10 13
After moving max to end: 29 13 14 10 37
After moving max to end: 14 13 10 29 37
After moving max to end: 13 10 14 29 37
After moving max to end: 10 13 14 29 37

Sorted Array: 10 13 14 29 37
```

THANK YOU!